

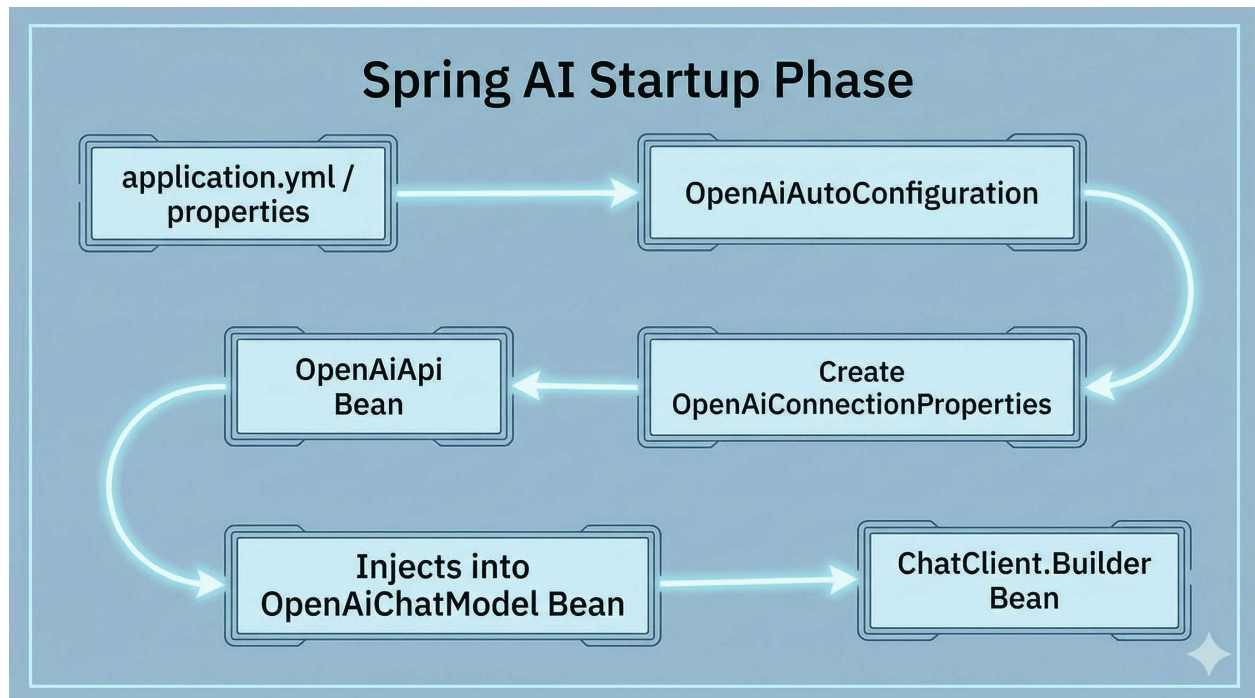
Introduction To Spring AI

1. Why Spring AI Exists vs. Raw HTTP Calls

- Building AI applications often starts with raw HTTP requests to providers like OpenAI. While simple initially, this approach becomes unmanageable as you scale. You end up hardcoding vendor-specific JSON bodies, managing authentication headers manually, and rewriting error handling logic for every different model provider (e.g., OpenAI's error format differs from Anthropic's).
- Spring AI solves this by providing a portable abstraction layer. It decouples your business logic from the specific AI implementation.
- Instead of writing `RestTemplate` calls for OpenAI and rewriting them for Azure, you write code against Spring AI interfaces.
- This allows you to swap the underlying AI provider (like switching from H2 to PostgreSQL in Spring Data) without changing your Java code. It handles the boilerplate of serialization, request formatting, and response parsing automatically.

2. Core Abstractions: From ChatClient to ChatModel

Spring AI has evolved its terminology to be more precise. The hierarchy you interact with is designed for flexibility.



- **ChatModel** : This is the low-level interface that represents a specific AI provider (e.g., `OpenAiChatModel`, `OllamaChatModel`). It knows how to talk to the specific API.
- **ChatClient** : This is the high-level, fluent API you will use 99% of the time. It wraps the `ChatModel` and provides a builder-style interface to compose prompts, attach advisors (memory, RAG), and handle responses. It acts as the "driver" for your AI interactions.
- **EmbeddingModel** : A separate interface dedicated to converting text into vector arrays (embeddings), which is essential for RAG (Retrieval-Augmented Generation) but distinct from chat generation.

3. Adding the Dependency (Spring AI 1.1.2)

To get started, you need the generic OpenAI starter. This brings in the autoconfiguration and the default `ChatModel` implementation. Add this to your

`pom.xml` :

```

<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>

```

```
<version>1.1.2</version>
</dependency>
```

4. Configuration via application.yml

Spring AI leverages Spring Boot's externalized configuration. You do not hardcode keys in Java. In your `src/main/resources/application.yml`, configure your API key and default model parameters. This global configuration applies to every call unless overridden.

```
spring:
  ai:
    openai:
      api-key: ${OPENAI_API_KEY} # Read from environment variable
    chat:
      options:
        model: gpt-4o
        temperature: 0.7
```

5. First "Hello World" with ChatClient (Fluent API)

The `ChatClient` is your primary entry point. You should inject the `ChatClient.Builder` (which is auto-configured by the starter) and build your client. The fluent API allows you to chain methods to construct your request.

```
@RestController
public class ChatController {

    private final ChatClient chatClient;

    public ChatController(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }
}
```

```

    @GetMapping("/hello")
    public String generate(@RequestParam(defaultValue = "Tell me a joke") String message) {
        return chatClient.prompt()
            .user(message)
            .call()
            .content();
    }
}

```

6. Converting to Structured Output (Entity Extraction)

LLMs generate text, but your application often needs Java objects. Spring AI's `.entity()` method handles this automatically. It instructs the model to produce JSON matching your schema and then parses it into your Java Record.

```

public record Joke(String setup, String punchline) {}

@GetMapping("/joke-json")
public Joke generateJoke() {
    return chatClient.prompt()
        .user("Tell me a funny programming joke.")
        .call()
        .entity(Joke.class);
}

```

7. Switching Providers (OpenAI → Azure → Ollama)

The power of Spring AI is portability. To switch from OpenAI to Ollama (local LLM), you only need to change the dependency and the configuration properties. Your Java code using `ChatClient` remains exactly the same.

- **Step 1:** Swap the dependency in `pom.xml`:

- Remove `spring-ai-starter-model-openai`
- Add `spring-ai-starter-ollama`
- **Step 2:** Update `application.yml`:

```
spring:
  ai:
    ollama:
      base-url: http://localhost:11434
      chat:
        options:
          model: llama3
```